האוניברסיטה העברית בירושלים

מספר מחברת ________

לפני הבחינה אנא מלא/מלאי את הפרטים בכתב ברור ובדייקנות

מספר תעודת הזהות | | | | | | | | | |

שם הקורס ____________ מס' הקורס ____________

שם המרצה ____________ תאריך הבחינה ____________

מקום להדבקת מדבקת ברקוד

<u>הנחיות לנבחן:</u>

- יש להכין תעודה מזהה.
- יש להישמע להוראות הבוחנים והמשגיחים.
- אסור לנבחנים לשוחח ולהתקשר בכל צורה ביניהם או עם כל גורם אחר, או להעביר חומר כלשהו זה לזה.
- אין לעזוב את אולם הבחינה במהלך הבחינה, אלא ברשות בוחן או משגיח.
- חל איסור להשתמש בבחינה בחומר כלשהו או להחזיקו למעט חומר שהותר לשימוש במפורש על ידי הבוחנים ובתנאי שאין בו כל רישומים, פתקים וכיו״ב. <u>כל החפצים האחרים ירוכזו באולם הבחינה בהתאם להוראות המשגיחים.</u>
- יש לכתוב את התשובות בעט <u>כחול או שחור</u> בכתב יד ברור ונקי. <u>אין</u> לכתוב בעיפרון.
- אין להשתמש בטיפקס או במרקרים צבעוניים.
- אין לכתוב בשוליים. יש לכתוב טיוטה רק על העמוד הפנימי של הדף במקום המסומן לכך.
- אין לתלוש או להוסיף דפים למחברת. מחברת שייתלשו או יצורפו אליה דפים דינה כמחברת פסולה.
- יש להקפיד למלא את כל הפרטים המזהים על גבי כל מחברת נוספת שאין עליה מדבקת ברקוד.
- יש למסור את המחברת בשלמותה לפני עזיבת האולם. עזיבת האולם ללא מסירת מחברת דינה ציון 0.
- **האוניברסיטה העברית מקפידה על קיומן התקין של הבחינות לפי הנהלים וכללי היושר הקבועים בתקנוניה, ומאחלת לך הצלחה בבחינות.**

<u>לשימוש המרצה</u>

ציון הבחינה (0-100) ____________ שם הבודק ____________

המחברת נבדקה בתאריך ____________ חתימת הבודק ____________

שים לב! השוליים יחתכו לפני הסריקה. לכן, חל איסור מוחלט לכתוב כאן.

מבוא למדעי המחשב (2 -67101) - סמסטר ב', מבחן מועד א' 2.7.21

מרצה הקורס: ד"ר אריה שלזינגר

במבחן זה יש לענות על כל 10 השאלות בחלק א׳, ולבחור 4 מתוך 6 השאלות בחלק ב׳ (אין לענות על יותר מ-4 שאלות בחלק ב׳). יש לענות על כל השאלות בגוף השאלון - בתוך ״קופסאות התשובה״ בלבד. אין להגיש דפי טיוטה - הם לא יבדקו. את התשובות יש לכתוב בעט שחור או כחול בלבד. אין לכתוב בשוליים.

חומר מותר לשימוש במבחן: אין. משך הבחינה : 3 שעות.

חלק א׳ - שאלות סגורות (40% - יש לענות על כל 10 הסעיפים, כל סעיף מזכה ב - 4 נק׳)

בחלק זה של המבחן יש לכתוב את התשובות הסופיות בלבד בתוך המשבצת המתאימה. אין צורך לנמק.

שאלה	תשובה
1	
2	
3	
4	
5	

שאלה	תשובה
6	
7	
8	
9	
10	

סמנו כאן על איזה שאלות בחרתם לענות **בחלק ב׳**, הקיפו בעיגול

1	**2**	**3**	**4**	**5**	**6**

חלק א׳

1. (4%) מה ידפיס קטע הקוד הבא:

```
t1, t2, liz = 10, 20, list(range(7))
a = min(filter(lambda x: x >= t1//2, liz))
b = max(filter(lambda x: x <= t2, liz))
print((lambda liz:(a,b))(liz))
```

---

2. (4%) מה ידפיס קטע הקוד הבא:

```
def f2(n):
    pn = n
    while pn:
        n, pn = n * 10 + pn % 10, pn // 10
    return n

print(list(map(f2, [1, 143])))
```

---

3. (4%) רשמו את סיבוכיות זמן הריצה של הפונקציה **f3**, כפונקציה של **n**:

```
def f3(n):
    for i in range(n):
        for j in range(n):
            for k in range(i + j):
                return f3(n-1)
```

---

4. (4%) רשמו את סיבוכיות זמן הריצה של הפונקציה **f4**, כפונקציה של **n**:

```
def f4(n):
    i = 1
    while i < n:
        j = i
        while j < n:
            while j < n:
                j += 1
            j += 1
        i += 1
```

---

5. (4%) מה יודפס בקריאות הבאות:

```
def excep(x):
    try:
        try:
            print(7/x)
            print(x != 0)
        except ZeroDivisionError:
            print("zero")
        except TypeError:
            print("type")
        finally:
            raise Exception(str(bool(x)))
    except Exception as m:
        print(m)

excep(0)
print(excep("0"))
```

---

6. (4%) מה יודפס בקריאות הבאות:

```
def guy(x):
    lst = [x]
    def ziv(y):
        lst.insert(0, y[0])
        print(lst)
        if y[1]:
            lst.pop()
    return ziv

adi = guy(3)
adi((0,2))
adi((1,0))
adi((None, None))
```

---

7. (4%) מה ידפיס קטע הקוד הבא:

```
def f7(n, k):
    res = 0
    st = str(n)
    if n > 0:
        x = 0
    else:
        x = 1
    for i in range(x, len(st) + 1):
        y = st[:i] + str(k) + st[i:]
        res = max(res, int(y))
    return res

print(f7(6674,5))
```

---

8. (4%) כתבו מה יודפס על ידי הרצת הקוד הבא:

```
def goo(i):
    if (i > 0):
        i -= 1
        goo(i)
        print(i, end=" ")
        i -= 1
        goo(i)

first = [0, 1, 2, 3, 4]
second = first[:]
first = first[::-1]
goo(second[4])
```

---

9. (4%) מה ידפיס קטע הקוד הבא:

```
def zoom(a, b):
    t = b[0][0]
    c = a
    c[0] = b[1]
    b = [c[:][1], c[:][1]]
    b[-1] = a[0][:]
    a[0][0] = t
    print(a)
    print(b)
    print(c)

lsty1 = [1,3]
lsty2 = [[4],[5]]
zoom(lsty1, lsty2)
```

---

10. (4%) מה ידפיס קטע הקוד הבא:

```
def gene(num, t):
    if num < 10:
        yield num
    else:
        yield from gene(num//t, t+1)
        yield num // t

print(list(gene(120, 1)))
```

חלק ב' – שאלות תכנות

(60% - יש לענות רק על **4 מתוך 6** השאלות, לכל שאלה 15 נק')

– יש לענות על כל שאלה רק בתוך קופסת הטקסט שניתנה בדף.

– מומלץ להשתמש במחברת הטיוטה כדי לגבש את הפתרון ורק כאשר הוא מושלם להעתיקו לטופס הבחינה.

– אין חובה לכתוב הערות או תיעוד.

– יש לכתוב קוד ברור ויעיל ככל הניתן (מבחינת זמן ריצה וזיכרון) על דפי התשובה של השאלה. אם יש צורך, ישנם דפים נוספים בסוף המבחן.

– אם לא נאמר מפורשות אחרת, ניתן להניח כי כל הקלטים לפונקציות הינם תקינים.

הניחו קיום המחלקה **Node** הבאה :

```
class Node:
    def __init__(self, data=None, next=None):
        self.data = data
        self.next = next
```

1. (15%) כתבו פונקציה (**linked_filter(f, lnk** אשר מקבלת פונקציה **f** ורשימה מקושרת **lnk**, ומחזירה רשימה מקושרת **חדשה** שתבנה כך: האיבר **k** מתוך **lnk**, יופיע ברשימה החדשה אם הערך של **f(k.data)** הוא **True**. הסדר בין האיברים ברשימה החדשה יהיה כמו ב-**lnk**. אם הרשימה המקושרת ריקה, הפונקציה תחזיר **None**. דוגמאות:

```
>> linked_filter(lambda x: (x+1)%2,Node(2,Node(3,Node(4,Node(5, Node(6))))))
Node(2, Node(4, Node(6)))
```

נסמן רשימה מקושרת ריקה על ידי - **None**

```
>> print(linked_filter(lambda x: (x+1) % 2,None))
None
```

def linked_filter(f, lnk):

2. (15%) כתבו את הגנרטור **all_sums(num, bound)** אשר מקבל שני מספרים שלמים עבורם מתקיימים: **num >= bound >= 1**.
בכל הפעלה (**next**), הגנרטור יחזיר מחרוזת אחרת אשר מתארת חיבור בין מספרים קטנים או שווים ל-**bound**, שסכומם הוא **num**. כל מחרוזת תוחזר פעם אחת בלבד.

לכל זימון של הגנרטור, יוחזרו כמחרוזות כל הסכומים האפשריים בהתאם לקלט.
דוגמאות (כאן התוצאות רוכזו ברשימה):

```
res = all_sums(4, 3)
list(res)
> ['3 + 1', '2 + 2', '2 + 1 + 1', '1 + 1 + 1 + 1']

res = all_sums(5, 2)
list(res)
> ['2 + 2 + 1', '2 + 1 + 1 + 1', '1 + 1 + 1 + 1 + 1']

res = all_sums(2, 2)
list(res)
> ['2', '1 + 1']

res = all_sums(1, 1)
list(res)
> ['1']
```

def all_sums(num, bound):

3. (15%) כתבו פונקציה רקורסיבית **f3(n, x, y)** אשר מקבלת מספר שלם לא שלילי **n**, ושני אובייקטים שונים **x, y**.
הפונקציה תחזיר רשימה של כל תת הרשימות השונות באורך **n**, שאיבריהן מבין **x**, ו-**y**, ובהן **y** אינו מופיע פעמיים ברצף. דוגמאות:

```
f3(3, 'a', (2,))
>
[ ['a', 'a', 'a'],
  ['a', 'a', (2,)],
  ['a', (2,), 'a'],
  [(2,), 'a', 'a'],
  [(2,), 'a', (2,)] ]

f3(2, [3,'5'], 'xy')
> [[[3, '5'], [3, '5']], [[3, '5'], 'xy'], ['xy', [3, '5']]]

f3(1, [3], '9')
> [[[3]], ['9']]
```

`def f3(n,x,y):`

4. (15%) כתבו פונקציה **f4(st, dict1, k)** אשר מקבלת כקלט:

- **st** - מחרוזת של אותיות.
- **dict1** - מילון הממפה לכל אות מספר שלם.
- **k** - מספר טבעי.

הפונקציה **f4** תחזיר תת-מחרוזת של **st** באורך **k**, כך שסכום ערכי התווים שלה יהיה מקסימלי (על פי המילון). תאור:
תת המחרוזת תיבנה כך שכל התווים שלה לא נמצאים כולם ברצף אחד ב-**st**, והסדר ביניהם ישאר כמו ב-**st**. דוגמאות:
אם **st** היא המחרוזת: **abcd**, תת המחרוזות המתאימות (שמביניהן בוחרים "מקסימלית") הן:

עבור **k=3**, אחת מבין: **abd,acd**

עבור **k=2**, אחת מבין: **ac,ad,bd** (ולא **ab**)

עבור **k=1** : תת המחרוזת יכולה להיות כל תו בודד מ **st** .

הניחו שהקלט ל **f4** תקין, ומתקיים: **len(st)>k>=1**. אם נמצאו כמה תת מחרוזות מתאימות, ניתן להחזיר אחת מהן. לדוגמא, הקלט הבא:

st = ‘abzzcd’, dict1 = {‘a’:1,‘b’:1,..,‘m’:1,‘n’:-1,‘e’:-1,..,’z’:-1}

בדוגמא, הערך של כל אות מ-**a** עד **m** ,הוא **1**, ושל כל אות מ-**n** עד **z** הוא **-1**.

עבור **k=2**, הערך המרבי הוא **2**. תת-המחרוזות המתאימות הן: **"bd", "ac", "ad", "bc"** ,
(אבל לא :"**ab**" או "**cd**")

עבור **k=3**, הערך המרבי הוא **3**. תת-המחרוזות המתאימות הן: **"bcd" , "abd", "abc"**.

עבור **k=5**, הערך המרבי הוא **4**, תת-המחרוזת המתאימה היא: "**abzcd**".(כל **z** מותרת)

```
def f4(st, dict1, k):
```

def f4(st, dict1, k):

5. (15%) כתבו פונקציה **rotate_90_clock(mat)** אשר מקבלת מטריצה לא ריקה **mat**, אשר מיוצגת כרשימה של רשימות. הרשימות הפנימיות מייצגות את שורות המטריצה לפי הסדר.

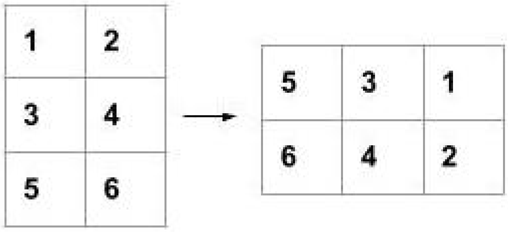
הפונקציה תחזיר מטריצה שהיא התוצאה של סיבוב ב-**90°** בכיוון השעון של **mat**. (כלומר השורה הראשונה תהיה העמודה האחרונה (הימנית ביותר), השורה השנייה תהיה העמודה הלפני האחרונה, וכו'), דוגמה, עבור המטריצה:

```
>> mat = [[1, 2], [3, 4], [5, 6]]
>> rotate_90_clock(mat)
```

נקבל:

```
>> mat
[[5, 3, 1], [6, 4, 2]]
```

המחשה:




```
def rotate_90_clock(mat):
```

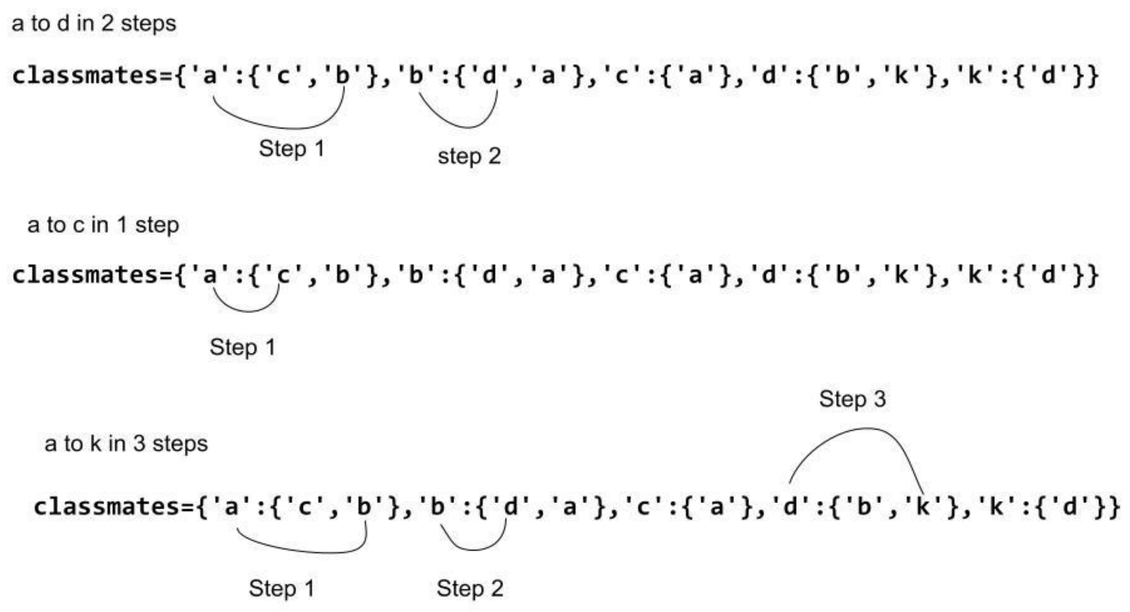
6. (15%) כתבו מחלקה **Campus** אשר האובייקטים שלה הם רשתות של חברים ללימודים בקמפוס. במחלקה יהיו שלוש שיטות:

1. השיטה **`__init__(self)`** יוצרת מילון ריק כמשתנה של האובייקט.

את המילון ממלאים רק באמצעות השיטה **`add_classmate`** (מתוארת בהמשך), באופן הבא:
מפתח במילון הוא שם של סטודנט, והערך שלו הוא קבוצה (**`set`**) בה שמות של כמה מחבריו ללימודים מכיתות שונות בהן הוא משתתף. ה-**`set`** אינו בהכרח מכיל את כל חבריו ללימודים של סטודנט מסוים.
כל השמות במילון הם מחרוזות.

2. השיטה **`add_classmate(self, st1, st2)`** מקבלת שמות של שני סטודנטים שלומדים באותה כיתה, ומוסיפה כל אחד מהם לקבוצת השמות (**set**) של חבריו ללימודים של הסטודנט השני. אם אחד מהם או שניהם אינם מופיעים במילון החברים, הוסיפו אותם למילון.

3. השיטה **`soc_dist(self, st1, st2, n)`** מקבלת שמות של שני סטודנטים, ומספר טבעי **`n`** , ומחזירה **True** אם **st1, st2** מופיעים במילון, וה״מרחק החברתי״ ביניהם הוא **n**. אם המרחק אינו **n**, או אם אחד מהם או שניהם לא נמצאים במילון, הפונקציה תחזיר **False**. בנוסף, אם **`st1`** ו-**`st2`** זהים, הפונקציה תחזיר **`True`** לכל **n** . המחשה:



נתאר ״מרחק חברתי״ בין שני סטודנטים **`st1,st2`** להיות מספר קבוצות החברים שיש לעבור כדי להגיע מ-**st1** כמפתח ל-**st2** כאיבר בקבוצה (**`set`**) כלשהיא. לדוגמה:

```
>teva = Campus()
>teva.add_classmate('a','b')
>teva.add_classmate('a','c')
>teva.add_classmate('d','b')
>teva.add_classmate('d','k')
>print(teva.classmates)
{'a': {'c', 'b'}, 'b': {'d', 'a'}, 'c': {'a'}, 'd': {'b', 'k'}, 'k': {'d'}}

>print("a-b",teva.soc_dist('a','b',1))
a-b True

>print("a-d",teva.soc_dist('a','d',2))
a-d True


>print("a-k",teva.soc_dist('a','k',3))
a-k True

>print("a-p",teva.soc_dist('a','p',3))
a-p False


>print("s-p",teva.soc_dist('s','p',1))
s-p False

>print("a-a",teva.soc_dist('a','a',100))
a-a True
```

class Campus:

דפי כתיבה נוספים:

ב ה צ ל ח ה